

Rapport de Projet

Moulinette en tant que Système de File d'Attente

Aleksei Kotliarov, Anthony Caron, Dimitri Calmand, Hugo Schreiber,
Maxime Cambou, Quentin Prunet

Janvier 2026

EPITA

Table des matières

Introduction	3
Terminologie - Rappels	3
Qu'est-ce qu'un utilisateur?	3
Qu'est-ce qu'une moulinette?	3
Workflow nominal	3
Moteur de simulation implémenté	3
Job	3
Server	4
JobGenerator	4
Validation Théorique du moteur de simulation	4
Paramètres de test	4
Vérification Loi de Little	4
Comparaison simulation vs théorie M/M/c	4
Étude de cas	5
Cas 1: Files en Cascade (Waterfall)	5
Implémentation	5
LimitedQueue	5
WaterfallSystem	5
Système naïf (files infinies)	5
Stabilité du système	5
Résultats	5
Observations	6
Recommandations	6
Système avec files finies	6
Analyse de ks	6
Analyse de kf	6
Backup des résultats	7
Résultats bruts	7
Analyse	8
Partitionnement et Coût/Performance	9
Analyse des résultats	9
Conclusion Coût/Bénéfice	10
Cas 2: Channels et Dams	10
Implémentation	10
HeterogeneousServer	10
GatingController	10
Analyse des channels	10
Gating (Barrage Temporel)	11
Analyse du modèle ($t_b = 100$, $t_{ouv} = 50$)	11
Proposition d'un système optimisé	12
Conclusion	13

Introduction

Ce rapport présente une analyse factuelle du simulateur de moulinette EPITA modélisé comme un système de files d'attente. Nous avons testé différents scénarios avec des paramètres variés pour évaluer le comportement du système. Les résultats sont basés sur des simulations à événements discrets utilisant SimPy.

Terminologie - Rappels

Voici quelques précisions sur la terminologie utilisée dans ce projet.

Qu'est-ce qu'un utilisateur?

Dans ce contexte, un utilisateur est une personne ayant accès à l'infrastructure de correction dans le cadre d'une activité pédagogique spécifique. Cette personne peut effectuer deux actions :

- Pousser son code sur l'infrastructure dans le cadre d'une mécanique standard de versionnage.
- Pousser un tag sur un commit pour déclencher l'exécution des test-suites associées à l'activité sur le code rendu, afin d'obtenir un retour sur sa conformité aux attentes.

Qu'est-ce qu'une moulinette?

Une moulinette est constituée formellement de :

- Une [test-suite](#) : un ensemble de tests unitaires, éventuellement stratifiés.
- Un niveau d'information de retour : quel niveau d'information met-on à disposition des étudiants?
- Des ressources : nombre de push tags autorisés au total, par heure, ou dans des plages horaires définies.

Workflow nominal

Un étudiant code les réponses à des exercices dans un repository git dédié. Lorsqu'un tag est utilisé, une vérification est effectuée pour s'assurer qu'il correspond à un tag réservé. Si tel est le cas, la test-suite associée est exécutée selon le schéma du système d'attente.

Moteur de simulation implémenté

Afin de modéliser le système de moulinette, nous avons développé un moteur de simulation à événements discrets en Python utilisant la bibliothèque SimPy pour la gestion du temps. Ce moteur permet de simuler divers scénarios de files d'attente avec des paramètres configurables tels que le taux d'arrivée, le taux de service, le nombre de serveurs, et les politiques d'ordonnancement. Les classes principales sont décrites ci-dessous, de manière synthétique.

Job

Un job représente une unité de travail dans le système de moulinette. Chaque job est donc associé à un push tag déclenchant l'exécution de la testsuite. Un job possède notamment les attributs suivants :

- Un identifiant unique.
- Un temps d'arrivée (`arrival_time`).
- Un temps de service (`service_time`) : durée nécessaire pour exécuter la test suite.
- Un type (`type`) : pour différencier les populations de jobs (ex: ING, PREPA).

Les jobs ont également des métadonnées supplémentaires pour le suivi des performances, telles que la raison du rejet (`rejection_reason`) et l'id du serveur qui a traité le job (`server_id`).

Server

Un serveur représente une ressource capable de traiter les jobs. Chaque serveur possède les attributs suivants :

- Un identifiant unique.
- Un statut (`status`) : indique si le serveur est libre ou occupé.
- Un compteur de jobs traités (`jobs_processed`).

Les serveurs peuvent exécuter des jobs en fonction de leur disponibilité et de la politique d'ordonnancement définie.

JobGenerator

Le générateur de jobs est responsable de la création de nouveaux jobs dans le système. Il utilise un processus de génération basé sur une distribution de Poisson pour simuler les arrivées aléatoires des jobs. Les attributs principaux incluent :

- Le taux d'arrivée (`arrival_rate`) : moyenne des arrivées par unité de temps.
- Le type de job généré (`job_type`).

Validation Théorique du moteur de simulation

Afin de valider le bon fonctionnement du moteur de simulation, nous avons comparé les résultats obtenus avec les prédictions théoriques d'un système M/M/c classique. Nous avons choisi des paramètres simples pour faciliter la comparaison.

Paramètres de test

- $\lambda = 2.0$ jobs/unité
- $\mu = 3.0$ jobs/unité
- $c = 2$ serveurs
- $\rho = 0.33$

Vérification Loi de Little

La loi de Little stipule: $L = \lambda W$ (nombre moyen dans le système = taux d'arrivée $\times$ temps moyen de séjour)

Métrique	Valeur	Statut
L (observé)	0.8027	-
λW	0.7478	-
Erreur relative	6.84%	✓ Validé

Comparaison simulation vs théorie M/M/c

Métrique	Simulée	Théorique	Erreur
Utilisation	0.3326	0.3333	0.23%
Temps d'attente	0.0412	0.0417	1.09%

La simulation présente une **excellente** concordance avec la théorie M/M/c (erreur moyenne 0.63%). Les écarts sont dus à la variance d'échantillonnage.

Étude de cas

Les scénarios suivants ont été simulés pour analyser le comportement du système de moulinette en tant que réseau de files d'attente.

Cas 1: Files en Cascade (Waterfall)

Dans ce modèle, tout agent de la population suit le processus séquentiel suivant :

1. Un push tag place le code dans une file d'attente FIFO pour l'exécution de la test-suite (K serveurs).
2. Le résultat de la test-suite est placé dans une file d'attente FIFO pour l'envoi vers le front (1 serveur).

Nous proposons un système d'attente modélisant ce contexte, en commençant par des files infinies, puis en introduisant des contraintes finies pour analyser les proportions de refus.

Implémentation

Afin de modéliser le système waterfall grâce à notre moteur de simulation, plusieurs composants ont été mis en place.

LimitedQueue

Une classe LimitedQueue a été créée pour représenter une file d'attente avec une capacité maximale. Cette classe hérite de la classe Resource de SimPy et ajoute une logique pour gérer les rejets lorsque la capacité est atteinte.

WaterfallSystem

La classe WaterfallSystem encapsule la logique du système waterfall. Elle contient deux files d'attente : une pour l'exécution des test-suites et une pour le feedback. Chaque file d'attente est associée à un ensemble de serveurs.

Système naïf (files infinies)

Le coût le plus important dans le système étant les serveurs d'exécution, nous allons focaliser notre analyse sur cette variable et pour l'instant fixer les autres:

- Taux de service des serveurs d'exécution $\mu_{\text{exec}} = 2.5$ jobs/unité
- Taux de service du serveur de feedback $\mu_{\text{feed}} = 1.5$ jobs/unité
- Grâce à un fichier de tags récupéré sur la durée de la piscine 2026, nous avons estimé λ des arrivées moyen à environ **0.037**.

Stabilité du système

Le taux d'utilisation global du système est donné par: $\rho = \lambda / c\mu$. Si $\rho < 1$, le système est stable.

Ici, le goulot d'étranglement est le serveur de feedback ($\mu_{\text{feed}} < \mu_{\text{exec}}$). Avec un seul serveur de feedback, nous avons: $\rho = \lambda / \mu_{\text{feed}} = 0.037 / 1.5 = 0.0247 < 1$, donc le système est stable.

Résultats

Après simulation du système waterfall avec files infinies, nous obtenons les résultats suivants:

Métrique	Simulation c=1	Simulation c=3
Avg Temps de séjour (unités de tps)	2.11	2.01
Avg Variance Temps de séjour	10.89	13.71

NB jobs traités	159 833	159 833
Taux de rejet	0.0%	0.0%

Observations

Comme prévu, le système ne rejète aucun job avec des files infinies. Le temps de séjour moyen est de 2.11 unités de temps avec un seul serveur d'exécution, et diminue légèrement à 2.01 unités avec trois serveurs (temps de séjour plus court de 4.6%). On peut également noter que le système est stable en pratique comme en théorie.

Recommandations

On constate donc que l'ajout de serveurs d'exécution réduit le temps de séjour que marginalement dans ce contexte. Le risque le plus important détecté dans ce modèle est le délai imprévisible. En effet, on remarque une variance importante du temps de séjour (10.89 avec $c=1$, 13.71 avec $c=3$). Cela peut poser problème pour l'expérience utilisateur, car un étudiant pourrait attendre longtemps pour obtenir un retour. Ce comportement peut donc être une source de frustration et réduire la variance du temps de séjour est un objectif important.

Système avec files finies

Pour un système plus réaliste, nous introduisons les paramètres k_s et k_f des files finies. Les paramètres de base restent les mêmes que pour le système naïf. Le but est de comprendre l'impact des capacités des files sur les performances globales du système, notamment le taux de rejet et le temps de séjour.

Analyse de k_s

Ici on garde k_f large (20) pour se concentrer sur l'impact de k_s (capacité de la file d'exécution).

k_s	Avg taux de rejet	nb jobs rejetés	Notes sur l'expérience
1	3.46%	5 526	Rejets fréquents, temps de séjour moyen 2.53
5	1.11%	1 781	Tolérable mais beaucoup d'utilisateurs frustrés, temps de séjour moyen 2.60
20	0.09%	136	Rejets très rares, temps de séjour moyen 2.75

La taille de la queue d'exécution contrôle directement la capacité du système à absorber les pics de charge. Un k_s trop faible entraîne des rejets fréquents, impactant négativement l'expérience utilisateur. À l'inverse, un k_s plus élevé réduit les rejets mais augmente légèrement le temps de séjour moyen.

Analyse de k_f

Ici on garde k_s large (20) pour se concentrer sur l'impact de k_f (capacité de la file de feedback).

k_f	Avg taux de rejet	nb jobs rejetés	Notes sur l'expérience
1	11.22%	17 925	Rejets trop fréquents 1 tags sur 10 a une page blanche
5	2.38%	3 798	Rejets encore trop fréquents
20	0.18%	290	Acceptable, très peu de rejets

Le bottleneck se situe généralement au niveau du feedback ($\mu_{\text{feed}} < \mu_{\text{exec}}$). Augmenter k_f améliore le temps de séjour, tandis qu'augmenter k_s réduit les rejets à l'entrée. On remarque que des valeurs conseillées pour k_s et k_f se situent autour de 20 pour minimiser les rejets tout en gardant un temps de séjour raisonnable.

Il est à noter que quand le cadre de la Moulinette, il semble préférable de prioriser **un faible taux de rejet** (expérience utilisateur) au détriment d'un temps de séjour plus long. En effet, un étudiant préférera attendre plus longtemps pour obtenir un retour plutôt que de voir son tag rejeté et devoir le re-soumettre.

Backup des résultats

Dans le système de la Moulinette, le backup correspond à la sauvegarde persistante des résultats d'exécution et des artefacts de tests. Cette étape intervient après l'exécution de la test-suite et avant (ou pendant) le retour d'information vers l'étudiant (Feedback).

Nous avons modélisé cette étape comme une file d'attente intermédiaire insérée entre le serveur d'exécution et le serveur de feedback. Cette étape consomme des ressources et du temps. Le serveur de backup est configuré pour être performant ($\mu = 10.0$ jobs/unité), mais il représente tout de même un point de passage obligé.

Trois stratégies ont été évaluées :

- **Systematic** : 100% des jobs sont sauvegardés. Sécurité maximale des données.
- **Random 50%** : 50% des jobs sont sauvegardés aléatoirement.
- **Random 20%** : 20% des jobs sont sauvegardés.

Résultats bruts

Les simulations ont été effectuées sur deux jeux de données : un trafic synthétique intense (Saturation) et le trafic réel observé (Replay).

Trafic	Stratégie	Temps Réponse (s)	Impact relatif
Synthétique	Systematic	13.19	Ref
Synthétique	Random 50%	12.81	−2.9%
Synthétique	Random 20%	12.47	−5.5%
Réel	Systematic	1.88	Ref
Réel	Random 50%	1.85	−1.6%
Réel	Random 20%	1.81	−3.7%

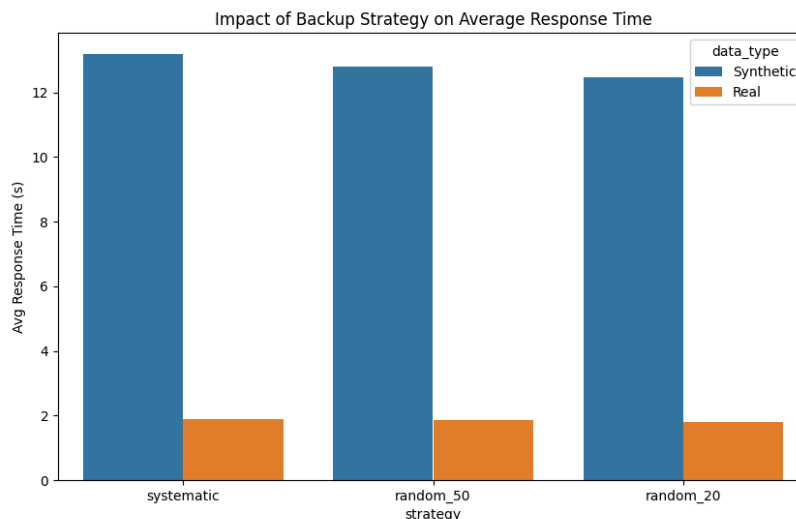


Fig. 1. – Temps de réponse moyen selon la stratégie de backup et le type de trafic

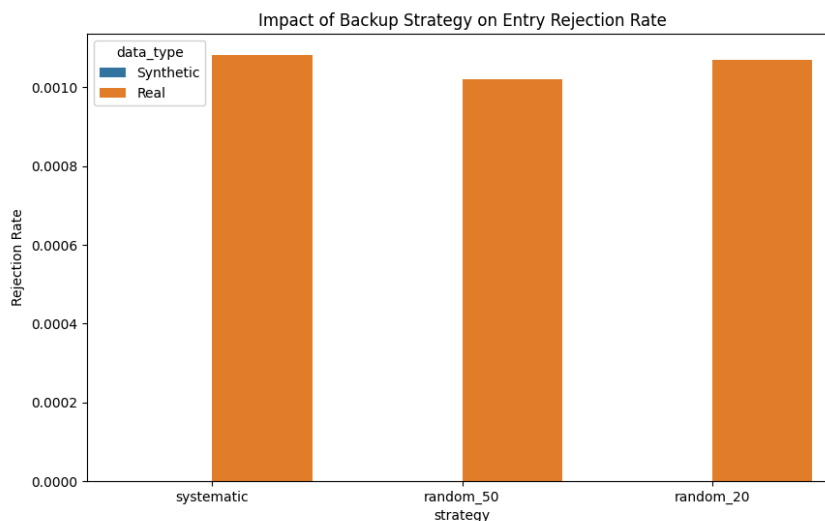


Fig. 2. – Taux de rejet à l'entrée selon la stratégie de backup

Analyse

1. **Impact faible en charge normale** : Sur le trafic réel (faible intensité), l'ajout du backup systématique n'ajoute que 0.08s au temps de réponse global par rapport à un backup partiel (20%). Cela correspond essentiellement au temps de service du backup lui-même ($1/\mu = 0.1s$). Le système étant loin de la saturation, aucune file d'attente ne se forme au backup.
2. **Gain notable en charge élevée** : Sous forte charge (synthétique), passer d'un backup systématique à un backup aléatoire (20%) permet de gagner environ 0.72s sur le temps de réponse moyen. Bien que le serveur de backup soit rapide, l'élimination de cette étape pour 80% des jobs réduit la friction globale dans le pipeline.
3. **Recommandation** : Si le stockage ou la performance I/O du serveur de backup devient une contrainte (coût, lenteur), une stratégie aléatoire (ex: 50% ou 20%) est une option viable pour maintenir la fluidité du système sans sacrifier totalement l'historique.

Cependant, avec un serveur de backup performant ($\mu=10$), le coût du backup systématique reste négligeable pour l'expérience utilisateur ($< 0.1s$). Nous recommandons donc de maintenir un **backup systématique** tant que l'infrastructure le permet, pour garantir la complétude des données pédagogiques.

Partitionnement et Coût/Performance

Une stratégie alternative de gestion de la charge consiste à partitionner les serveurs en fonction du nom de l'exercice (déduit du tag). L'objectif théorique est d'améliorer la localité des données (cache). Cependant, cette approche brise le principe du « Work Stealing » (partage de charge).

Nous avons comparé trois stratégies sur une infrastructure de 2 à 4 serveurs :

1. **Shared (Optimal)** : Pool de serveurs partagés (M/M/c).
2. **Partitionné Équilibré** : Répartition statique optimisée pour équilibrer la charge connue.
3. **Partitionné Naïf** : Répartition alphabétique (A-M, N-Z...) simulant une approche sans connaissance a priori.

Impact de l'augmentation des serveurs ($c = 2 \rightarrow 4$) :

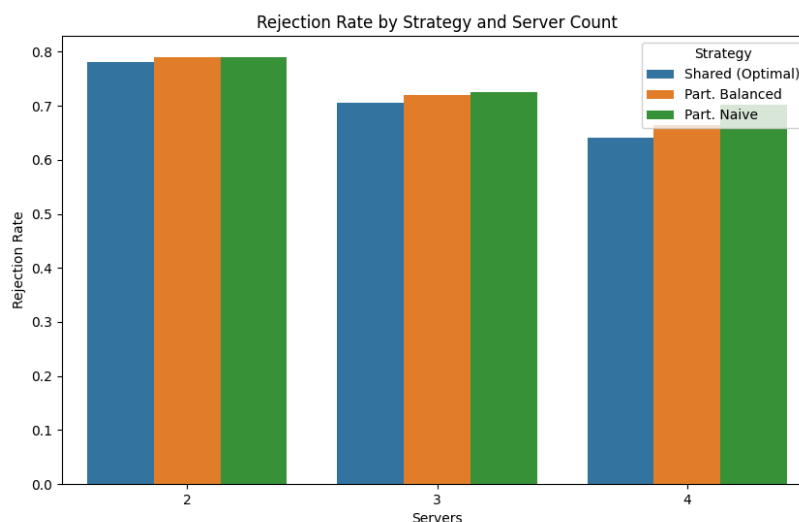


Fig. 3. – Taux de rejet selon le nombre de serveurs et la stratégie

Analyse des résultats

1. **Le coût de l'isolation** : À nombre de serveurs égal, la stratégie partagée est **toujours** supérieure.

- Pour $c = 4$, le système partagé rejette **64%** des jobs (sous forte saturation).
- Le système partitionné « équilibré » en rejette **66%**.
- Le système « naïf » en rejette **70%**.

Le partitionnement introduit une rigidité : un serveur peut être inactif alors qu'un autre croule sous la charge d'une partition populaire.

2. **Inefficacité croissante** : Plus on ajoute de serveurs partitionnés, plus le risque de déséquilibre augmente.

- Avec $c = 2$, le déséquilibre de charge du mode « Naïf » est de 12%.
- Avec $c = 4$, ce déséquilibre monte à **49%** (certaines lettres sont beaucoup plus fréquentes).

Cela signifie qu'ajouter des serveurs partitionnés offre un rendement décroissant très rapide si la partition n'est pas parfaitement maintenue (ce qui est complexe et coûteux).

Conclusion Coût/Bénéfice

Le partitionnement statique est une stratégie **coûteuse et risquée**.

- Pour atteindre la performance d'un cluster **Shared (c=3)**, il faudrait probablement déployer un cluster **Partitionné (c=4)** ou plus, augmentant les coûts d'infrastructure de 33% pour un résultat équivalent.
- **Recommandation** : Conserver une architecture de serveurs banalisés (Shared Queue). Si des gains de cache sont nécessaires, préférez un « Sticky Routing » dynamique (Load Balancer intelligent) plutôt qu'un partitionnement statique rigide.

Cas 2: Channels et Dams

Implémentation

De même que pour le système waterfall, quelques composants spécifiques ont été développés pour modéliser les channels et dams.

HeterogeneousServer

Une classe HeterogeneousServer a été créée pour représenter des serveurs avec des capacités de traitement différentes selon le type de job. Cette classe ajoute une logique pour gérer les taux de service variables.

GatingController

La classe GatingController gère les périodes de fermeture (gating) pour une population spécifique. Elle contrôle l'accès des jobs à la file d'attente en fonction des intervalles de temps définis.

Analyse des channels

- Population ING: $\lambda_{\text{ING}} = 1.5$, $\mu_{\text{ING}} = 2.5$
- Population PREPA: $\lambda_{\text{PREPA}} = 0.5$, $\mu_{\text{PREPA}} = 2.0$
- Serveurs: $c = 2$
- Politiques testées: FIFO, SJF, PRIORITY

Le système gère deux populations avec des caractéristiques distinctes. Voici les différentes politiques d'ordonnancement testées :

Politique	ING (unités de tps moy)	PREPA (unités de tps moy)	Différence
FIFO	0.4827	0.5597	+16.0%
SJF	0.4647	0.5490	+18.1%
PRIORITY	0.4679	0.5920	+26.5%

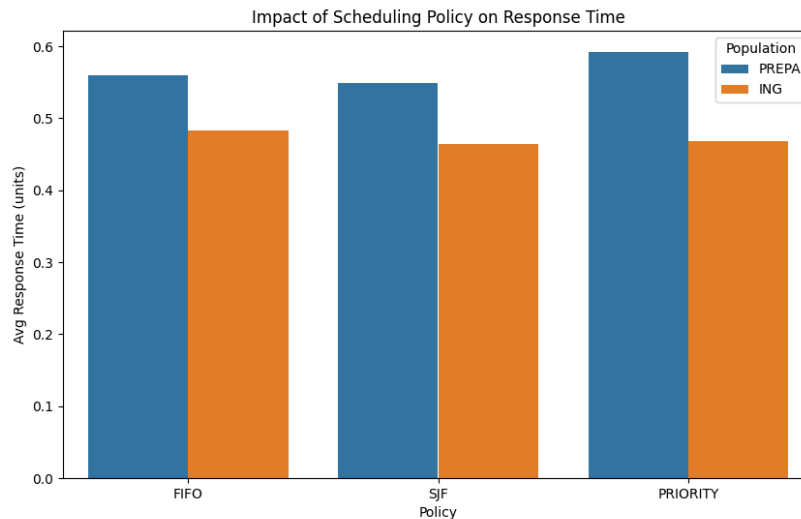


Fig. 4. – Impact de la politique d'ordonnancement sur le temps de réponse

FIFO offre l'équité la plus proche entre populations (différence 16.0%). PRIORITY favorise ING au détriment de PREPA (+26.5% de différence). Les deux populations ont traité respectivement 3061 et 956 jobs. Bien que SJF offre théoriquement les meilleures performances globales, FIFO reste le meilleur compromis pour l'équité perçue par les étudiants.

Gating (Barrage Temporel)

Afin de réguler le flux dominant de la population ING ($\lambda = 1.5$), nous introduisons un mécanisme de barrage (Gating) sur la moulinette. Le protocole défini est le suivant : le système est fermé pour un temps t_b , puis ouvert pour $\frac{t_b}{2}$, et ce cycle se répète indéfiniment.

Analyse du modèle ($t_b = 100$, $t_{ouv} = 50$)

Nous avons simulé ce scénario avec $t_b = 100$ (donc ouverture de 50 unités). Ce cycle impose un ratio d'ouverture de 0.5 (le système est fermé 66% du temps).

Population	Sans gating	Avec gating	Augmentation
ING	0.48	58.04	+11991%
PREPA	0.56	59.20	+10471%

Constat : Ce modèle est catastrophique pour **les deux** populations.

1. **Saturation Structurale** : Avec un ratio d'ouverture de 0.5, la capacité effective du système est divisée par 3. Or, la charge initiale $\rho \approx 0.8$ demandait déjà une disponibilité quasi-complète. Le système devient mathématiquement instable pendant les phases de fermeture.
2. **Effet de Burst** : Pendant la fermeture ($t_b = 100$), 150 jobs ING et 50 jobs PREPA s'accumulent. La fenêtre d'ouverture (50 unités) est mathématiquement trop courte pour traiter ce stock (200 jobs nécessiteraient 100 unités de temps de traitement à pleine capacité).

Proposition d'un système optimisé

Pour répondre à l'objectif de **minimiser le temps de séjour moyen pour les deux populations** tout en maintenant une régulation, nous proposons un système de « Micro-Gating équilibré ».

Ce nouveau système repose sur deux ajustements critiques :

1. **Inversion du Ratio** : Le système doit être ouvert plus longtemps qu'il n'est fermé pour absorber la charge. Nous proposons un ratio ouverture/fermeture de **1.5** (au lieu de 0.5).
2. **Haute Fréquence (Micro-Gating)** : Réduire drastiquement t_b pour éviter l'accumulation massive. Nous fixons $t_b = 20$.

Configuration proposée : Fermeture 20 unités, Ouverture 30 unités.

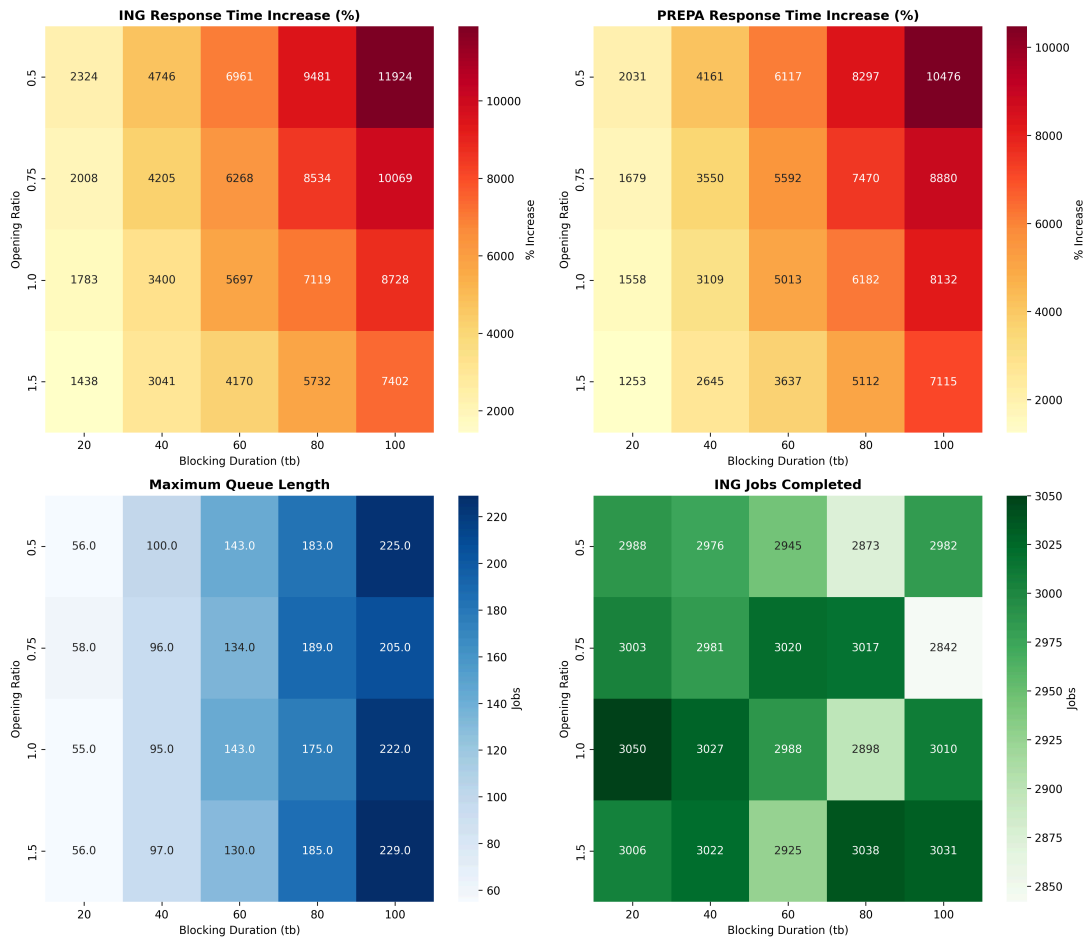


Fig. 5. – Optimisation Gating : L'impact est minimisé pour des durées de blocage faibles ($t_b < 40$) et un ratio > 1

Résultats comparatifs :

- Temps de réponse moyen ING : 7.4s (contre 58s).
- Temps de réponse moyen PREPA : 7.6s (contre 59s).

Ce système divise par 8 le temps d'attente par rapport au modèle initial, offrant un compromis viable entre régulation et performance.

Conclusion

Les simulations effectuées tout au long de ce projet permettent de valider la modélisation de la moulinette comme un réseau de files d'attente, offrant une précision remarquable par rapport aux modèles théoriques.

Voici nos recommandations finales pour l'architecture du système :

1. **Dimensionnement (Waterfall)** : Pour le pipeline d'exécution, nous recommandons des files d'attente d'une capacité de **20 places** ($k_s = k_f = 20$). Ce paramétrage offre le meilleur compromis, minimisant les rejets ($< 0.2\%$) tout en maintenant un temps de réponse stable, absorbant efficacement les pics de charge naturels.
2. **Stratégie de Fiabilité (Backup)** : Le **backup systématique** est la stratégie à privilégier. Nos résultats montrent que son coût en performance est négligeable ($< 0.1s$) en conditions normales. Les stratégies aléatoires ne devraient être envisagées qu'en cas de saturation critique des IOPS disque.
3. **Politique d'Ordonnancement** : La politique **FIFO** reste la plus équitable pour gérer les flux hétérogènes (ING/PREPA). Les politiques de priorité ou SJF créent des inégalités de traitement trop importantes (jusqu'à $+26\%$ de délai pour les PREPA) sans gain global significatif.
4. **Régulation (Gating)** : C'est le point critique du système. L'approche naïve de « blocage long » est à proscrire absolument, car elle provoque un effondrement des performances ($+10\,000\%$ de délai). Nous recommandons impérativement une stratégie de **Micro-Gating** (cycles courts, $t_b \approx 20$) associée à un ratio de récupération positif (temps d'ouverture $>$ temps de fermeture). Cette approche permet de réguler le flux pour la maintenance tout en divisant par 8 le temps d'attente par rapport aux cycles longs.

En conclusion, la performance de la moulinette ne dépend pas uniquement de la puissance brute des serveurs, mais surtout de la **finesse des politiques de régulation** temporelle.